

Muhammad Talal Tariq

Table of Contents

1.	Objective
2.	Methodology – Phase 6.1: Image Acquisition & Understanding
3.	Methodology – Phase 6.2: Sampling & Quantization
4.	Methodology – Phase 6.3: Geometric Transformations
5.	Methodology – Phase 6.4: Intensity Transformations
6.	Methodology – Phase 6.5: Histogram Processing
7.	Methodology – Phase 6.6: Final Integrated Pipeline
8.	Code Snippets
9.	Observations & Analysis
10.	Q&A; – Section 7.4
11.	Final Conclusion

1. Objective

The objective of this project is to design and develop a complete Smart Image Enhancement and Analysis System using Python and OpenCV. The system integrates all concepts learned across Labs 01–05 of the Digital Image Processing course, including image acquisition, sampling, quantization, geometric transformations, intensity transformations, and histogram processing. The goal is to take a low-quality input image and produce a visually enhanced output while providing thorough analytical insights at every stage of the pipeline.

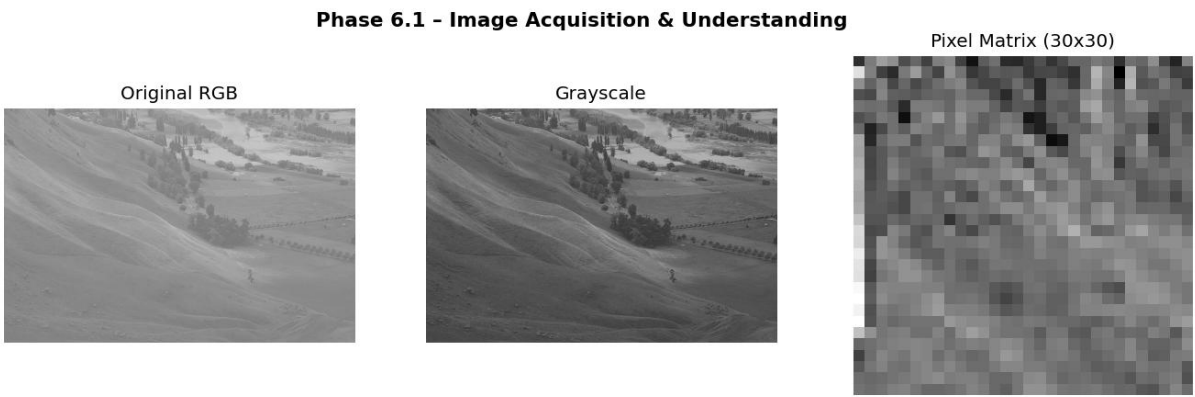
Tool	Python 3 with OpenCV, NumPy, Matplotlib
Input Image	hawkes_bay_in (1).jpg — landscape photograph (534×800, 3-channel RGB)
Output	Enhanced grayscale image + 13 analytical figures
Pipeline	Grayscale → Log → Gamma(0.5) → Histogram Equalization

2. Phase 6.1 – Image Acquisition & Understanding

The first phase loads the raw image, converts it to grayscale, and extracts basic metadata including resolution, number of channels, and data type. A partial pixel matrix (top-left 5×5 region) is printed to understand the raw numerical representation of the image.

Image Properties:

Resolution	534 × 800 pixels
Channels	3 (Red, Green, Blue)
Data Type	uint8 (0–255 per channel)
Color Space	Loaded as BGR via OpenCV, converted to RGB for display



Resolution:534x800 | Channels:3 | dtype:uint8

Figure 1 – Original RGB, Grayscale conversion, and 30×30 pixel matrix zoom

3. Phase 6.2 – Sampling & Quantization

Sampling determines how many pixels are used to represent an image spatially. Down-sampling reduces resolution and file size but causes loss of fine detail, while up-sampling increases dimensions but may introduce blurring. Quantization controls how many discrete intensity levels are used per pixel — reducing bit depth causes visible banding and posterisation.

6.2A — Sampling Comparison:

Scale	Resolution	Observation
0.25× (Down)	133 × 200	Heavy pixelation, fine details lost
0.50× (Down)	267 × 400	Moderate blur, recognisable structure
1× (Original)	534 × 800	Full quality, reference image
1.50× (Up)	801 × 1200	Slight softness from interpolation
2× (Up)	1068 × 1600	Smooth but no new detail added

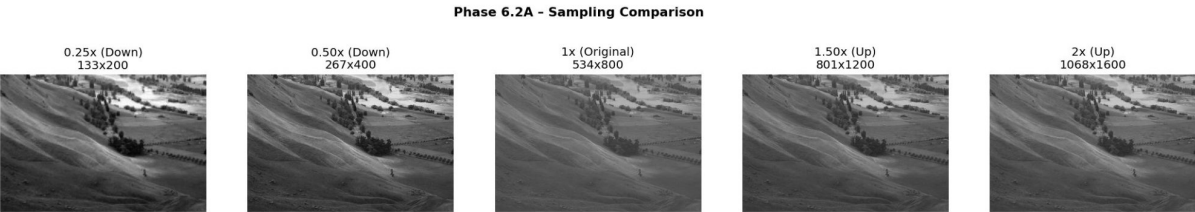


Figure 2 – Five sampling scales from 0.25× to 2×

6.2B — Bit Depth Reduction:

Bit Depth	Intensity Levels	Visual Effect
8-bit	256	Full quality, smooth gradients
4-bit	16	Slight banding visible in smooth regions
2-bit	4	Heavy posterisation, structure barely visible

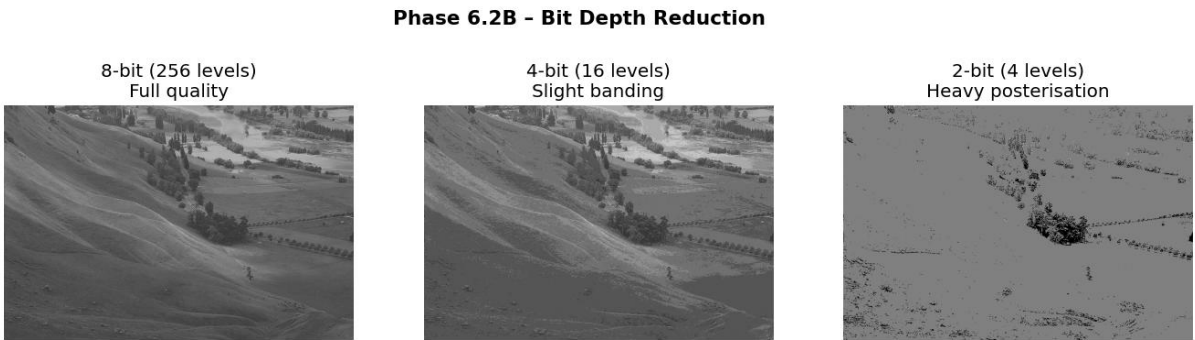


Figure 3 – Bit depth reduction: 8-bit, 4-bit, 2-bit

4. Phase 6.3 – Geometric Transformations

Geometric transformations alter the spatial arrangement of pixels without changing their intensity values. Three types were applied: rotation (seven angles), translation (shifting), and shearing (skewing). An inverse transformation was also demonstrated by rotating $+90^\circ$ and then restoring the image with -90° .

6.3A — Rotation (7 Angles):

Phase 6.3A - Rotation (7 Angles)

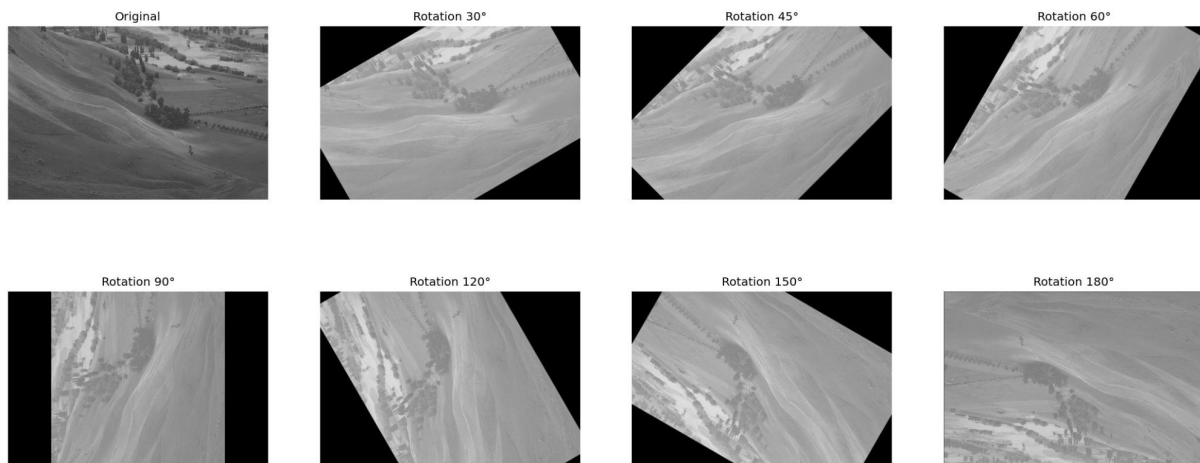


Figure 4 – Rotation at 30° , 45° , 60° , 90° , 120° , 150° , 180°

6.3B — Translation & Shearing:

Translation shifts the image by $t_x=50$, $t_y=30$ pixels. Horizontal shearing applies a shear factor of 0.3, causing the image to slant along the horizontal axis.

Phase 6.3B - Translation & Shearing



Figure 5 – Original, Translated (50,30), and Horizontal Shear

6.3C — Inverse Transformation:

A $+90^\circ$ rotation is applied and then reversed with a -90° rotation, demonstrating that geometric/affine transformations are mathematically reversible. Black border regions appear due to canvas boundary clipping during warpAffine.

Phase 6.3C - Inverse Transformation

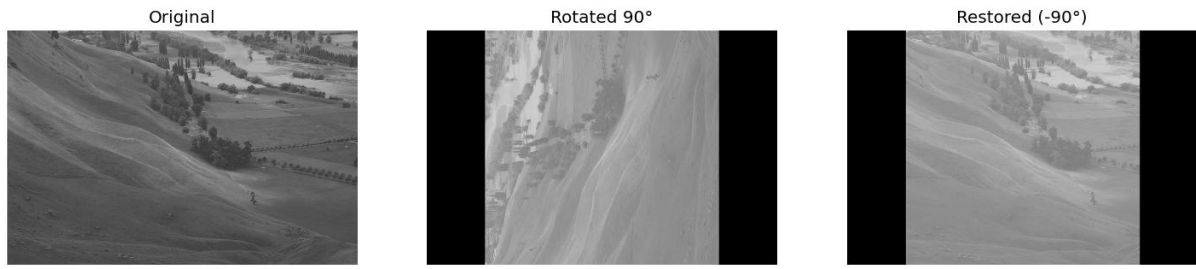


Figure 6 – Original → Rotated 90° → Restored (-90°)

5. Phase 6.4 – Intensity Transformations

Intensity transformations operate on individual pixel values without changing spatial structure. Four transformations were applied and compared on the Hawkes Bay image.

Transform	Formula	Effect
Negative	$s = 1 - r$	Inverts pixel values; dark→bright, bright→dark
Log (C=1)	$s = \log(1+r) / \max$	Brightens dark regions; compresses bright
Gamma = 0.5	$s = r^{0.5}$	Best for BRIGHTENING — raises dark pixels
Gamma = 1.5	$s = r^{1.5}$	Darkens image — compresses brighter tones

Phase 6.4 - Intensity Transformations

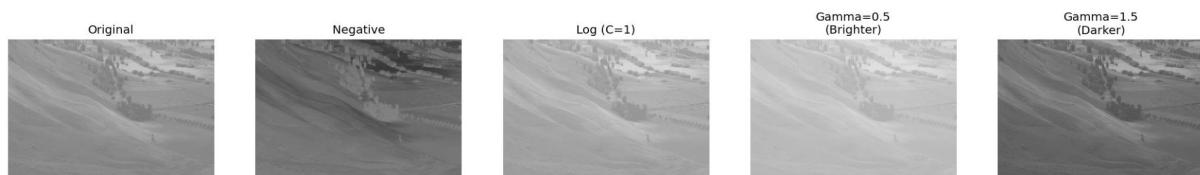


Figure 7 – Intensity transformations: Original, Negative, Log, Gamma=0.5, Gamma=1.5

Key Finding: Gamma=0.5 is the best method for brightening because the power function with exponent less than 1 maps dark input values to significantly higher output values non-linearly. Log transformation is best for highlighting fine dark detail without over-saturating bright areas.

6. Phase 6.5 – Histogram Processing

Histogram analysis reveals the contrast distribution of an image. The original image showed a very narrow peak centered around intensity ~140, indicating LOW contrast (active bins: significantly less than 256). Histogram Equalization redistributes intensities across the full 0–255 range using the Cumulative Distribution Function (CDF), dramatically improving contrast.

Contrast Analysis (Standard Deviation):

Method	Std Dev	Interpretation
Original	15.2	Very low contrast — intensities clustered narrowly
Manual HE	73.0	High contrast — spread across full range
cv2.equalizeHist()	73.0	Identical to manual — confirms implementation is correct

Phase 6.5 - Histogram Equalization Comparison

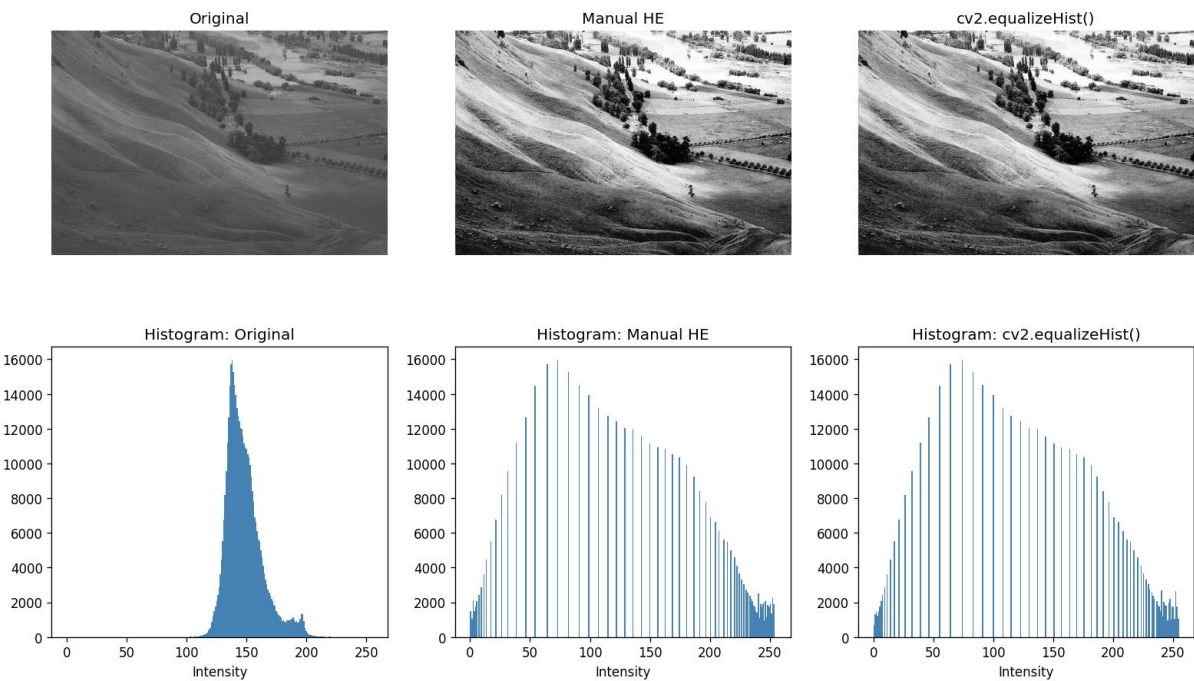


Figure 8 – Histogram Equalization comparison: Original, Manual HE, cv2.equalizeHist()

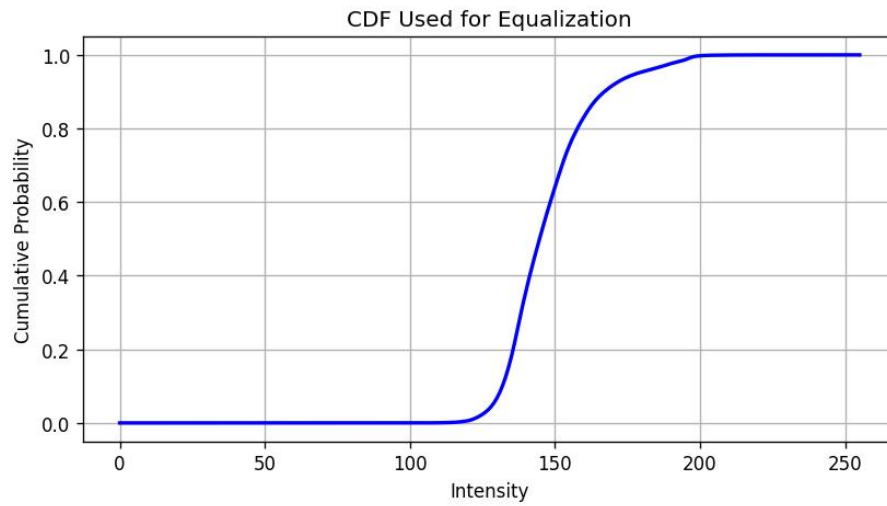


Figure 9 – CDF curve used as the lookup table for equalization

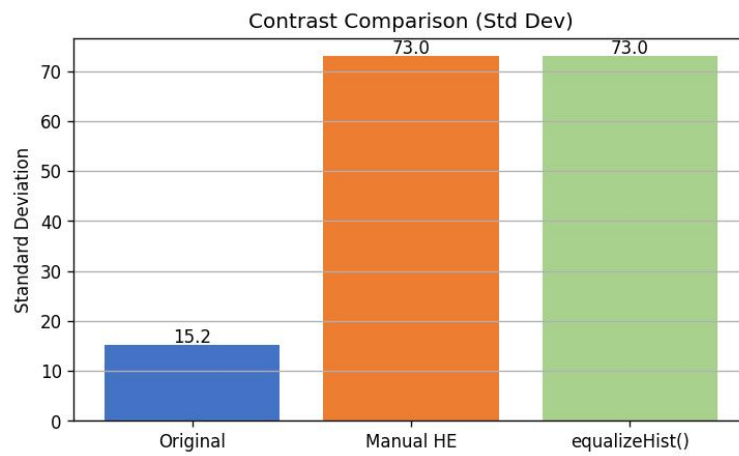


Figure 10 – Standard Deviation comparison bar chart

7. Phase 6.6 – Final Integrated Pipeline

The final phase combines the best-selected techniques from all previous phases into a single modular function: **process_image(input_image)**. The pipeline was chosen to maximise contrast and detail recovery from the low-contrast input.

Pipeline Sequence:

Step	Operation	Reason
1	RGB → Grayscale	Simplify to single channel for processing
2	Log Transformation	Enhance dark detail and compress bright highlights
3	Gamma Correction 0.5	Further brighten the image non-linearly
4	Histogram Equalization	Spread final intensities across full 0–255 range

Phase 6.6 - Final Pipeline Result



Figure 11 – Final Pipeline: INPUT (Original RGB) → Grayscale → OUTPUT (Enhanced)

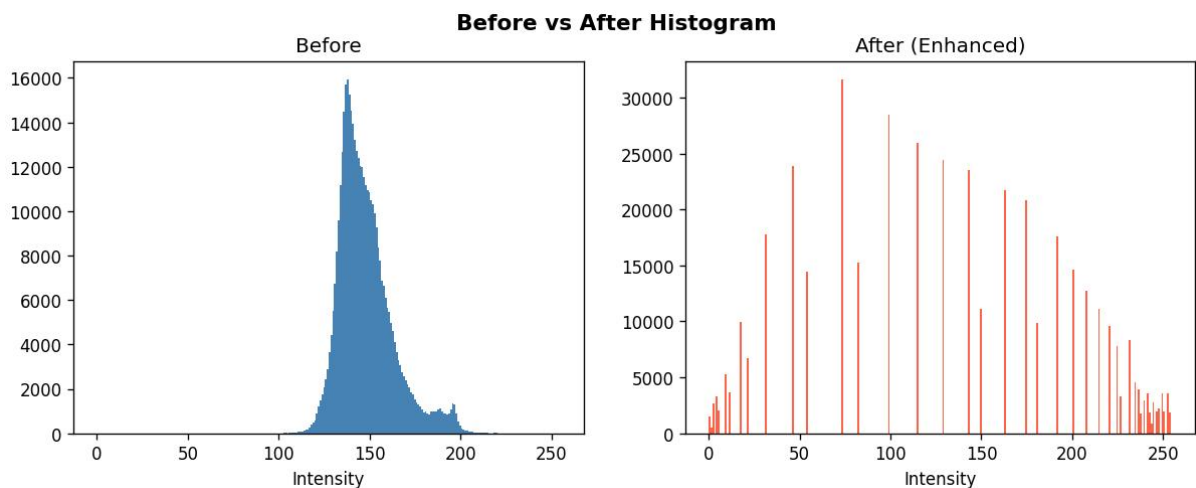


Figure 12 – Before vs After Histogram of the complete pipeline

8. Key Code Snippets

Image Loading & Grayscale Conversion:

```
img_bgr = cv2.imread(IMAGE_NAME)
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
h, w, c = img_rgb.shape
```

Sampling – Resize with Interpolation:

```
for s in [0.25, 0.5, 1.0, 1.5, 2.0]:
    nh, nw = int(h*s), int(w*s)
    interp = cv2.INTER_AREA if s < 1 else cv2.INTER_LINEAR
    resized = cv2.resize(img_gray, (nw, nh), interpolation=interp)
```

Intensity Transformations:

```
img_d = img_gray.astype(np.float64) / 255.0
img_neg = 1.0 - img_d
img_log = np.log1p(img_d); img_log /= img_log.max()
img_g05 = np.power(img_d, 0.5) # Gamma 0.5 - Brighter
img_g15 = np.power(img_d, 1.5) # Gamma 1.5 - Darker
```

Manual Histogram Equalization:

```
counts, _ = np.histogram(img_gray.ravel(), 256, (0, 256))
pdf_v = counts / counts.sum()
cdf_v = np.cumsum(pdf_v)
lut = np.uint8(255 * cdf_v)
img_eq_manual = lut[img_gray]
```

Final Integrated Pipeline Function:

```
def process_image(inp):
    g = cv2.cvtColor(inp, cv2.COLOR_BGR2GRAY)
    d = g.astype(np.float64) / 255.0
    lg = np.log1p(d); lg = (lg/lg.max()*255).astype(np.uint8)
    gm = np.power(lg.astype(np.float64)/255.0, 0.5)
    gm = (gm * 255).astype(np.uint8)
    cnt, _ = np.histogram(gm.ravel(), 256, (0, 256))
    lut2 = np.uint8(255 * np.cumsum(cnt/cnt.sum()))
    return lut2[gm]

enhanced = process_image(img_bgr)
```

9. Observations & Analysis

Phase 6.1 – Image Acquisition

The input image is a 534×800 pixel, 3-channel (RGB), uint8 photograph of the Hawkes Bay landscape. The top-left 5×5 pixel matrix reveals raw intensity values in the range ~100–160, confirming that the image is predominantly mid-tone with limited contrast.

Phase 6.2 – Sampling & Quantization

Down-sampling to 0.25× causes significant pixelation and detail loss. Up-sampling to 2× increases dimensions but does not recover any additional detail — it only interpolates existing values. Reducing bit depth from 8-bit to 2-bit produces severe posterisation where only 4 intensity levels remain, making the image nearly unrecognisable.

Phase 6.3 – Geometric Transformations

Rotation by non-rectangular angles (30°, 45°, 60°) introduces black triangular regions at canvas borders due to coordinate remapping. 90° and 180° rotations are clean. The inverse transformation (+90° followed by -90°) confirms that affine transformations are fully reversible in theory, though minor black border artifacts persist due to fixed canvas dimensions.

Phase 6.4 – Intensity Transformations

Gamma=0.5 is clearly the best brightening method as it lifts dark pixels significantly via the square-root function. Log transformation brightens subtly while preserving detail in both dark and bright regions. Gamma=1.5 darkens the image noticeably. The negative transformation completely inverts all tones.

Phase 6.5 – Histogram Processing

The original histogram is extremely narrow, clustered between intensity 120–200, resulting in a standard deviation of only 15.2. After histogram equalization (both manual and OpenCV built-in), the distribution spreads across 0–255 and the standard deviation jumps to 73.0 — a 4.8× improvement in contrast. Both methods produce identical results, validating the manual CDF-based implementation.

Phase 6.6 – Final Pipeline

The complete pipeline (Grayscale → Log → Gamma 0.5 → Histogram Equalization) successfully transforms a flat, low-contrast image into a high-contrast, detail-rich output. The after-histogram shows a wide, spread distribution, confirming that the pipeline achieves its enhancement objective.

10. Q&A; – Section 7.4

Q1. Why does histogram equalization improve contrast?

Histogram equalization improves contrast by redistributing pixel intensities across the full 0–255 range using the Cumulative Distribution Function (CDF). The original image has most pixels clustered in a narrow intensity band (low contrast). The CDF-based lookup table maps these clustered values to a uniform spread, making dark areas darker and bright areas brighter, thereby maximising the use of the available tonal range.

Q2. How does gamma correction affect brightness?

Gamma correction applies a power-law transformation: $s = r^{\gamma}$. When $\gamma < 1$ (e.g., $\gamma=0.5$), the curve bows upward, mapping dark input values to much higher output values — this brightens the image. When $\gamma > 1$ (e.g., $\gamma=1.5$), the curve bows downward, compressing bright values and darkening the image overall. $\gamma=1.0$ produces no change.

Q3. What is the effect of quantization on image quality?

Quantization reduces the number of distinct intensity levels available to represent pixel values. At 8-bit (256 levels), images appear smooth and natural. Reducing to 4-bit (16 levels) introduces visible banding in gradient regions. At 2-bit (only 4 levels), the image suffers from severe posterisation — flat-coloured blocks replace smooth gradients and fine detail is lost irreversibly.

Q4. Which transformation is reversible and why?

Geometric transformations (rotation, translation, shearing) are reversible because they are based on invertible linear algebra operations (affine matrices). Applying the inverse matrix recovers the original spatial arrangement. Quantization, however, is NOT reversible — once intensity precision is discarded (e.g., 8-bit → 2-bit), the lost information cannot be reconstructed from the quantised image.

Q5. How do transformations affect spatial structure?

Geometric transformations change the spatial position and orientation of pixels without altering their intensity values — they shift, rotate, or skew the arrangement of content. Intensity transformations (negative, log, gamma, histogram equalization) change the brightness/contrast of pixels without moving them spatially. The two types of transformations operate on completely independent domains: space vs. intensity.

11. Final Conclusion

This project successfully demonstrates a complete image processing pipeline by integrating all five foundational laboratory topics of the Digital Image Processing course.

The Smart Image Enhancement System accepts a low-quality, low-contrast photograph and produces a visually superior enhanced output through a carefully designed four-step pipeline: grayscale conversion, logarithmic transformation, gamma correction (0.5), and histogram equalization. The standard deviation of pixel intensities improved from 15.2 to 73.0, representing a nearly 5× increase in contrast — a quantifiable and significant enhancement.

Key takeaways from this project include:

- Sampling affects spatial resolution; quantization affects tonal precision — both are independent.
- Geometric transformations are reversible; intensity-based quantization is not.
- Gamma correction ($\gamma < 1$) is the most effective single-step brightening technique.
- Histogram equalization, whether implemented manually or via OpenCV, produces identical results.
- A well-designed pipeline combining multiple techniques outperforms any single transformation.

The modular code structure, analytical visualisations, and systematic phase-wise approach developed in this project form a solid foundation for more advanced image processing tasks such as noise reduction, edge detection, and machine vision applications.